

STR

1. Contexte et mise en situation

La Société de Tir de Reims (STR), fondée en 1884 et affiliée à la FFTir, est un club sportif accueillant environ 450 adhérents. Le club souhaitait une refonte complète de son site web pour moderniser sa communication en ligne et proposer un espace réservé aux adhérents.

Ce dossier couvre la conception complète du site : modélisation de la base de données, architecture applicative en PHP natif, implémentation des mesures de sécurité, et développement des fonctionnalités métier (gestion des comptes liée aux licences FFTir, blog avec éditeur WYSIWYG, back-office d'administration).

2. Objectifs

- Concevoir la base de données relationnelle (10 tables, 2 événements MySQL automatisés)
- Développer le front-office public (présentation du club, articles, contact)
- Développer l'espace adhérent avec contenu réservé
- Développer le back-office d'administration (gestion des comptes, articles, statistiques)
- Implémenter 6 mesures de sécurité applicative (OWASP Top 10)
- Réaliser les tests d'intégration, fonctionnels et de sécurité

3. Environnement technique

Système d'exploitation	Windows 11
Serveur local	WAMP 3.4.0 (Apache, MySQL 8.4.7)
Langage serveur	PHP 8.3.28 (natif, sans framework)
Base de données	MySQL 8.4.7 – InnoDB – utf8mb4
Langages front-end	HTML5, CSS3, JavaScript (vanilla)
Éditeur de code	Visual Studio Code
Gestion de version	Git / GitHub
Navigateur de test	Microsoft Edge + DevTools
Bibliothèque JS	Quill.js (éditeur WYSIWYG)
URL de développement	http://localhost/str_website/public/

4. Conception de la base de données

Le schéma relationnel comporte 10 tables InnoDB en utf8mb4, centré autour de la table users qui est liée à 6 autres tables par clés étrangères.

4.1 — Création de la table users

La table users utilise un champ ENUM account_status à 4 valeurs (pending, active, expired, rejected) au lieu d'un simple booléen is_active. Ce choix permet de gérer le cycle de vie complet d'un compte adhérent.

```
CREATE TABLE users (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  role_id     TINYINT NOT NULL,  
  email       VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  first_name  VARCHAR(100) NOT NULL,
```

```

last_name    VARCHAR(100) NOT NULL,
phone        VARCHAR(20) DEFAULT NULL,
account_status ENUM('pending','active','expired','rejected')
              NOT NULL DEFAULT 'pending',
created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (role_id) REFERENCES roles(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Le choix de l'ENUM `account_status` est déterminant : un booléen `is_active` ne permet pas de distinguer un compte en attente de validation d'un compte refusé ou d'un compte expiré. Les 4 états couvrent l'intégralité du cycle de vie métier.

4.2 — Table des licences (relation 1-N)

La table `licences` stocke les dates de validité et permet aux événements MySQL de détecter les comptes expirés.

```

CREATE TABLE licences (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  user_id         INT NOT NULL,
  licence_number  VARCHAR(50) NOT NULL,
  season_label    VARCHAR(20),          -- ex: '2025-2026'
  starts_at       DATE NOT NULL,
  expires_at      DATE NOT NULL,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

4.3 — Événements MySQL automatisés

Deux événements planifiés s'exécutent automatiquement chaque nuit pour gérer le cycle de vie des comptes sans intervention manuelle :

```

-- Désactive les comptes dont TOUTES les licences sont expirées
UPDATE users u
INNER JOIN roles r ON r.id = u.role_id AND r.code = 'adherent'
SET u.account_status = 'expired'
WHERE u.account_status = 'active'
AND NOT EXISTS (
  SELECT 1 FROM licences l
  WHERE l.user_id = u.id
  AND l.expires_at >= CURDATE()
)

-- Supprime les comptes expirés depuis plus de 2 mois
DELETE u FROM users u
INNER JOIN roles r ON r.id = u.role_id AND r.code = 'adherent'
WHERE u.account_status = 'expired'
AND NOT EXISTS (
  SELECT 1 FROM licences l
  WHERE l.user_id = u.id
  AND l.expires_at >= DATE_SUB(CURDATE(), INTERVAL 2 MONTH)
)

```

L'automatisation par événements MySQL évite de dépendre d'un cron système ou d'un script PHP planifié. La logique reste au plus près des données.

5. Architecture applicative

L'application suit une architecture MVC simplifiée en PHP natif, organisée en 4 couches. Le tableau ci-dessous décrit le rôle de chaque dossier :

public/	Pages accessibles par le navigateur : index, login, register, actualite, article, le-club, disciplines, contact, mon-compte, mentions-legales, confidentialite + sous-dossier admin/
----------------	--

handlers/	7 fichiers PHP de traitement des formulaires (POST) selon le pattern PRG (Post-Redirect-Get)
src/	Logique métier : db.php (connexion PDO + BASE_URL), auth.php (contrôle d'accès), csrf.php, mail.php
includes/	Composants réutilisables : header.php, header-admin.php, footer.php

5.1 — Connexion PDO sécurisée (src/db.php)

Le fichier db.php centralise la connexion à la base de données. Trois options PDO renforcent la sécurité et la cohérence des données :

```
$pdo = new PDO(
    'mysql:host=localhost;dbname=str_db;charset=utf8mb4',
    'root',
    '',
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

EMULATE_PREPARES = false force MySQL à préparer réellement les requêtes côté serveur. C'est une protection supplémentaire contre les injections SQL.

5.2 — Contrôle d'accès (src/auth.php)

Quatre fonctions centralisées gèrent l'accès. Elles sont appelées en tête de chaque page protégée :

```
function isLoggedIn(): bool
{
    return isset($_SESSION['user_id']);
}

function hasRole(string $role): bool
{
    return isset($_SESSION['role']) && $_SESSION['role'] === $role;
}

function requireLogin(): void
{
    if (!isLoggedIn()) {
        header('Location:' . BASE_URL . '/login.php');
        exit;
    }
}
```

```
function requireRole(string $role): void
{
    if (!hasRole($role)) {
        header('Location:' . BASE_URL . '/login.php');
        exit;
    }
}
```

Exemple d'utilisation sur une page admin :

```
// admin/users.php
requireRole('admin');
//redirige vers login.php si non admin
```

6. Sécurité applicative

Le site implémente 6 mesures de protection couvrant les vulnérabilités du OWASP Top 10. Voici une sélection de 4 protections.

6.1 — Injection SQL : requêtes préparées PDO

Aucune variable utilisateur n'est concaténée dans une requête SQL. 100% des requêtes utilisent des placeholders :

```
// handle-register.php
$stmt = $pdo->prepare('SELECT id FROM users WHERE email = ?');
$stmt->execute([$email]);
```

6.2 — XSS : echappement a l'affichage

Toutes les données affichées dans le HTML sont échappées. L'échappement est appliqué à l'affichage uniquement, jamais au stockage :

```
<input type="text" name="first_name" id="first_name" value="<?=
htmlspecialchars($old['first_name'] ?? '') ?>" required>
```

6.3 — CSRF : token a usage unique

Chaque formulaire contient un token généré puis vérifié côté serveur avec hash_equals() :

```
// csrf.php
function generateCsrfToken(): string
{
    $token = bin2hex(random_bytes(32));
    $_SESSION['csrf_token'] = $token;
    return $token;
}
```

```
function verifyCsrfToken(string $token): bool
{
    if (!isset($_SESSION['csrf_token'])) {
        return false;
    }

    $valid = hash_equals($_SESSION['csrf_token'], $token);
    unset($_SESSION['csrf_token']);
    return $valid;
}
```

6.4 — Hashage bcrypt + protection de session

Les mots de passe sont hashés en bcrypt cost 12. L'identifiant de session est régénéré à chaque connexion pour empêcher la fixation de session :

```
// handle-register.php

$password_hash = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);
$stmt = $pdo->prepare('
INSERT INTO users (first_name, last_name, email, password_hash, phone)
VALUES (?, ?, ?, ?, ?)
');
```

```
// à la connexion    handle-login.php
session_regenerate_id(true);
```

7. Traitement des formulaires (pattern PRG)

Tous les formulaires suivent le pattern Post-Redirect-Get : le handler traite le POST, stocke les messages en session, puis redirige vers la page d'origine. L'utilisateur ne peut pas re-soumettre le formulaire en rafraîchissant la page.

7.1 — Connexion : vérification du statut

Le handler de connexion vérifie le mot de passe puis redirige selon le statut du compte. Le message d'erreur est volontairement identique pour un email inexistant et un mauvais mot de passe (sécurité) :

```
if (!password_verify($password, $user['password_hash'])) {
    $_SESSION['form_errors'] = ['Email ou mot de passe incorrect.'];
    header('Location: ' . BASE_URL . '/login.php');
    exit;
}

if ($user['account_status'] === 'pending') {
    $_SESSION['form_errors'] = ['Votre compte est en attente de validation par le club.'];
}
```

```

    $_SESSION['form_old'] = ['email' => $email];
    header('Location: ' . BASE_URL . '/login.php');
    exit;
}
if ($user['account_status'] === 'expired') {
    $_SESSION['form_old'] = ['email' => $email];
    header('Location: ' . BASE_URL . '/renouvellement.php');
    exit;
}
if ($user['account_status'] === 'rejected') {
    $_SESSION['form_errors'] = ['Votre inscription a été refusée.'];
    $_SESSION['form_old'] = ['email' => $email];
    header('Location: ' . BASE_URL . '/login.php');
    exit;
}

```

Si le compte est actif, la session est créée et l'utilisateur est redirigé selon son rôle : admin ou personnel vers le dashboard, adhérent vers l'accueil.

```

//réussite
session_regenerate_id(true);
$_SESSION['user_id']    = $user['id'];
$_SESSION['role']       = $user['role_code'];
$_SESSION['first_name'] = $user['first_name'];
$_SESSION['last_name']  = $user['last_name'];

if ($user['role_code'] === 'admin' || $user['role_code'] === 'personnel') {
    header('Location: ' . BASE_URL . '/admin/dashboard.php');
} else {
    header('Location: ' . BASE_URL . '/index.php');
}
exit;

```

8. Gestion des comptes adhérents

La gestion des comptes est la fonctionnalité métier centrale du site. Elle est liée au cycle de vie des licences FFTir et comporte 4 états :

pending	Inscription soumise, en attente de validation par l'admin
active	Licence validée, accès complet au contenu réservé
expired	Licence expirée, accès bloqué, redirection vers renouvellement

8.1 — Validation d'un compte (AJAX)

L'administrateur gère les comptes via une modale AJAX dans admin/users.php. Le détail d'un adhérent est chargé dynamiquement :

```
// admin/user-detail.php

// protection AJAX : code 403 au lieu de redirection

if (!hasRole('admin')) {

    http_response_code(403);

    exit;

}
```

Bug critique corrigé : requireRole() faisait une redirection HTML dans un contexte AJAX. Le JS recevait le HTML de login dans la modale. Remplace par http_response_code(403).

8.2 — Calcul automatique de la saison FFTir

Lors de la validation, la saison est calculée automatiquement. La saison FFTir va d'octobre à septembre :

```
$now = new DateTime();

$year = (int)$now->format('n') >= 10 ? (int)$now->format('Y') : (int)$now->format('Y') - 1;

$season = $year . '-' . ($year + 1);

$stmt = $pdo->prepare('UPDATE licences SET season_label = ?, starts_at = ?, expires_at = ?
WHERE user_id = ? AND season_label = ""');

$stmt->execute([$season, $year . '-10-01', ($year + 1) . '-09-30', $user_id]);
```

9. Éditeur d'articles Quill

Les articles sont rédigés avec l'éditeur WYSIWYG Quill.js. Le contenu est stocké sous deux formes : content_delta (format JSON natif de Quill, pour la réédition) et content_html (rendu HTML, pour l'affichage) :

```
// JavaScript – Avant soumission du formulaire

document.getElementById('article-form').addEventListener('submit', function(){

    // Stocke le Delta JSON pour pouvoir recharger l'éditeur

    document.getElementById('content-delta').value=JSON.stringify(quill.getContents());

    // Stocke le HTML pour l'affichage public

    document.getElementById('content-html').value = quill.root.innerHTML;
```

La génération du slug URL est automatique, avec gestion des accents et dédoublement par timestamp :

```
// handle-publish.php    PHP – Génération du slug

$slug = mb_strtolower($title);

$slug = preg_replace('/[àâä]/u', 'a', $slug);

$slug = preg_replace('/[éèêë]/u', 'e', $slug);

$slug = preg_replace('/[îï]/u', 'i', $slug);

$slug = preg_replace('/[ôö]/u', 'o', $slug);

$slug = preg_replace('/[ùûü]/u', 'u', $slug);
```

```
$slug = preg_replace('/[ç]/u','c',$slug);
$slug = preg_replace('/[^a-z0-9]+/','-',$slug);
$slug = trim($slug, '-');

// Déduplication si le slug existe déjà
$stmt = $pdo->prepare('SELECT id FROM posts WHERE slug = ?');
$stmt->execute([$slug]);
if ($stmt->fetch()) $slug .= '-' . time();
```

10. Conclusion

Ce projet couvre l'intégralité du cycle de développement d'une application web en PHP natif : analyse du besoin métier (gestion des licences FFTir), conception d'une base de données relationnelle de 10 tables avec automatisation par événements MySQL, développement full-stack (31 fichiers PHP, 3 fichiers JavaScript, 1 fichier CSS), et implémentation de 10 mesures de sécurité couvrant le OWASP Top 10.

La spécificité métier, le cycle de vie des comptes lié aux licences FFTir avec ses 4 statuts et ses événements MySQL automatisés, illustre la capacité à traduire une règle de gestion complexe en solution technique fiable et maintenable.

Une évolution est déjà identifiée : suite à des conflits d'utilisation constatés au stand, un système de réservation de la cible électronique devra être développé, avec une priorité accordée à l'école de tir et aux compétiteurs.